End-to-End Data Management Pipeline for a Recommendation System

Name Divya Sharma **ID** 2025AG05573 **Dataset** MovieLens 1M **Repository** github.com/2025ag05573/DM4MLAssignment2

Abstract. This report describes a reproducible pipeline that ingests MovieLens 1M, validates it against a data contract, prepares and transforms it, serves point-in-time features from a small feature store, tunes and trains five recommenders with MLflow tracking, evaluates them on a future time window with bootstrap confidence intervals and runs end-to-end as one Prefect flow. On the test window implicit ALS and the hybrid beat popularity by **+15%–16% NDCG@10** (95% intervals exclude zero) and cover **2.5×** more of the catalogue.

1 PROBLEM, DATA AND ARCHITECTURE

The task is to recommend movies a user will like from their past ratings and item information. **MovieLens 1M** has all three required entities: 1,000,209 explicit ratings (1–5) from 6,040 users on 3,883 movies (Apr 2000–Feb 2003), with user demographics (gender, age bucket, occupation, ZIP) and movie titles and genres. It is large enough for realistic results, yet the full pipeline, including tuning, runs in about 10 minutes (reference run, one CPU-only machine).

Concern	Choice	Why
Storage	Parquet, layered raw → validated → clean → derived → features	Typed, columnar, no server; every layer can be inspected
Validation	Pandera schemas + custom pandas checks	Contracts as code; lighter than Great Expectations
Features	Custom file-based feature store	Feast's core ideas (registry, entities, point-in-time snapshots) without its infrastructure
Models	NumPy / SciPy ALS implementations	No compiled dependencies, so it installs anywhere; fully transparent
Tracking	MLflow (SQLite backend)	Params, metrics, data checksum, feature version, model artifacts
Orchestration	Prefect 3	Pure Python, runs with or without a server, retries, concurrency, schedules
Quality	pytest (23 tests), one YAML config, fixed seeds, lock file	Validation rules, checksums, metrics, leakage and models tested

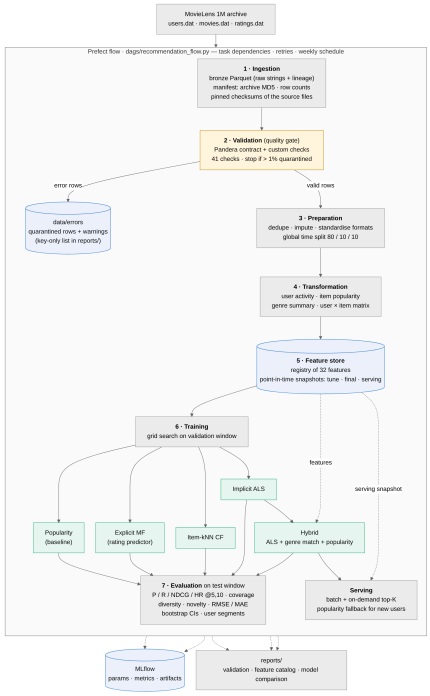


Figure 1. Pipeline architecture. Every stage is a Prefect task that reads and writes files, so each can be re-run on its own.

2 DESIGN DECISIONS

2.1 *Ingestion and validation*

Source files land as *bronze* Parquet with every column kept as a string plus lineage columns, so type problems are caught by validation instead of being silently coerced on read. A manifest records the archive MD5 and the MD5 of each source file, which are checked against pinned values (here: users match, movies match, ratings match; a mismatch is logged, or stops the run with `checksum_policy: fail`). Validation runs **41 checks** in three groups: a Pandera contract per table (types, ranges, allowed values, formats); integrity checks (missing keys, un-parseable values, duplicate keys, referential integrity from ratings to users and movies); and plausibility checks (timestamps after MovieLens launched and not in the future, ratings not before the movie's release year, genre vocabulary, row volumes). Each check has a severity. **Error** rows are quarantined to `data/errors/`, tagged with the checks they failed, and listed by id in the committed `quarantined_records.csv`. **Warning** rows are kept, logged, and repaired in preparation. A quality gate stops the flow if more than 1% of any table is quarantined. On the real data, 0 rows were quarantined and 1 check raised warnings: 81 users have ZIP+4 or malformed ZIP codes (66 truncated to 5 digits, 15 set to *unknown*). Because MovieLens is clean, the error path is shown by unit tests and by a demo report in which the same code quarantines 10 corrupted rows (nulls, duplicate keys, a rating of 7, future and pre-1997 timestamps, orphan ids, non-numeric values).

2.2 *Preparation and time-based split*

Repeated (user, movie) ratings keep the latest one. Missing categorical values become *unknown*. Timestamps become UTC datetimes, coded age and occupation become labels, and titles are split into title and release year with trailing articles moved to the front ("Matrix, The (1999)" → "The Matrix", 1999). Genres are one-hot encoded. The split uses a **single global time cut-off**, so nothing used for training happens after anything used for testing. A random or per-user split would leak future popularity trends into training. Train is the first 80% of ratings by time (to 2000-12-02), validation is the next 10% (to 2000-12-29) and is used only for tuning, and test is the last 10%. Final models are refitted on train + validation.

MovieLens activity is concentrated in 2000, so the test window spans two years and contains 1,209 users, 1,180 of whom have training history.

2.3 *Transformation and feature management*

Derived datasets (user activity summary, item popularity, genre summary, user×item rating matrix) are pure functions that take an `as_of` timestamp. The feature builder reuses them, so analysis, training and serving share one definition of, for example, "item popularity". The registry (`definitions.py`) holds **32 features** (14 user, 11 item, 7 interaction; the two genre families expand to 18 columns each). Each feature records its entity, type, source, transformation and consumers, and the catalog table is generated from the registry. Features are materialised point-in-time as three snapshots: *tune* (as of the validation start), *final* (as of the test start) and *serving* (after the last rating). The build fails if the materialised columns drift from the registry. Training and the inference CLI both read through one `FeatureStore` class; `ui_genre_match` is computed on demand.

2.4 *Models, tuning and tracking*

- **Popularity** (baseline): the most-rated unseen items. Its rating prediction is the damped (Bayesian) item mean.
- **Explicit MF**: biased matrix factorisation fitted with ALS and weighted-λ regularisation (64 factors, λ=0.1). It predicts ratings, which gives RMSE and MAE. Plain ridge is kept as a reference trial (validation RMSE 0.8613 vs 0.8595).
- **Item-kNN**: item-based collaborative filtering, cosine similarity of co-rated movies with shrinkage (shrink 200, 500 neighbours per item).
- **Implicit ALS**: every rating is a positive signal with confidence 1+α·rating, solved with a vectorised conjugate-gradient ALS (8 factors, λ=500, α=3).
- **Hybrid**: standardised ALS score + 0.4·genre-match + 0.5·standardised log-popularity. The content and popularity terms come from the feature store.

Hyper-parameters were grid-searched on the validation window only (NDCG@10 for ranking models, RMSE for explicit MF), with ranges wide enough that every selected value lies inside its grid. ALS uses a two-stage search: λ × α at 32 factors, then the number of factors at the best pair, keeping the smallest model within 2% of the best score (8 factors, NDCG@10 0.1443 vs best 0.1450). Strong regularisation is what makes ALS work here: with small λ it ranks below popularity. All 73 trials are nested MLflow runs. Every final model has its own run with parameters, input checksums, feature-registry version (`c57ff8074c25`), fit time and diagnostics, the pickled model and its test metrics.

2.5 *Orchestration and reproducibility*

A Prefect flow with 14 tasks: ingest (with retries) → validate (quality gate) → prepare → split → transform → features → tune → five training tasks (the independent models run concurrently and the hybrid waits for ALS) → evaluate → batch recommendations. Validation and evaluation summaries are published as Prefect artifacts, and `--serve` registers a weekly retraining schedule. One YAML config, fixed seeds, a deterministic split, pinned input checksums and a lock file with every package version make runs repeatable: the same metrics were reproduced on Python 3.11 (minimum dependency versions) and 3.12 (locked versions). Serving scores users the models were trained on; the 29 users whose first rating falls after the training window, and unknown ids, get the popularity fallback.

3 RESULTS

Protocol: 1,158 warm users with at least one relevant test item (rating ≥ 4). Candidates are the 3,678 items seen in the fit window, minus each user's already-rated items. The 29 cold-start users are excluded and reported. Coverage is the share of candidates recommended to anyone; diversity is the mean pairwise genre-cosine distance within a list; novelty is the mean $-\log_2$(popularity share). Confidence intervals use a paired bootstrap over users. RMSE and MAE are computed on 95,723 held-out ratings.

Model (K = 10)	Precision	Recall	NDCG	Hit rate	Coverage	Diversity	Novelty	RMSE	MAE
Popularity (baseline)	0.186	0.052	0.201	0.624	0.040	0.661	8.6	0.955	0.757
Explicit MF (biased ALS)	0.050	0.017	0.044	0.309	0.086	0.660	**15.2**	**0.891**	**0.697**
Item-kNN CF	0.190	0.053	0.209	0.602	0.058	0.656	8.9	—	—
Implicit ALS	0.211	0.066	0.232	0.659	**0.101**	**0.679**	8.9	—	—
Hybrid (ALS + features)	**0.215**	**0.068**	**0.233**	**0.678**	0.085	0.663	8.9	—	—



Figure 2. Test-window metrics at K = 10 (highlighted in the table = best per column). K = 5, confidence intervals and per-segment results are in `reports/evaluation/model_comparison.md`.

- **Personalisation helps.** Implicit ALS improves on popularity by +14% precision, +27% recall and +15% NDCG@10 (paired 95% bootstrap interval of the gain +0.0223 to +0.0400), beats it in every user-history segment and raises coverage from 4.0% to 10.1% of the catalogue. Item-kNN gains +4.0% NDCG@10, which is not significant (-0.0002 to +0.0161).
- **Popularity is a strong baseline here** (hit rate 62%): the 100 most-rated titles, 2.7% of the catalogue, still receive 12.8% of all test-window ratings.
- **Low RMSE does not mean good rankings.** Explicit MF has the best RMSE (0.891 vs 0.955; difference -0.081 to -0.049, users resampled) and MAE but the worst NDCG (0.044). Ranking by predicted rating surfaces rarely rated titles with high estimated means (novelty 15.2 vs 8.6 bits).
- **The hybrid's gain is marginal** (+0.6% NDCG over ALS, far inside the confidence intervals; hit rate 0.678 vs 0.659) and lowers diversity (0.663 vs 0.679) because it favours each user's dominant genres. Collaborative signals already capture most of the genre information. It is kept because it shows features being reused in training and serving, and because its score components explain each recommendation.

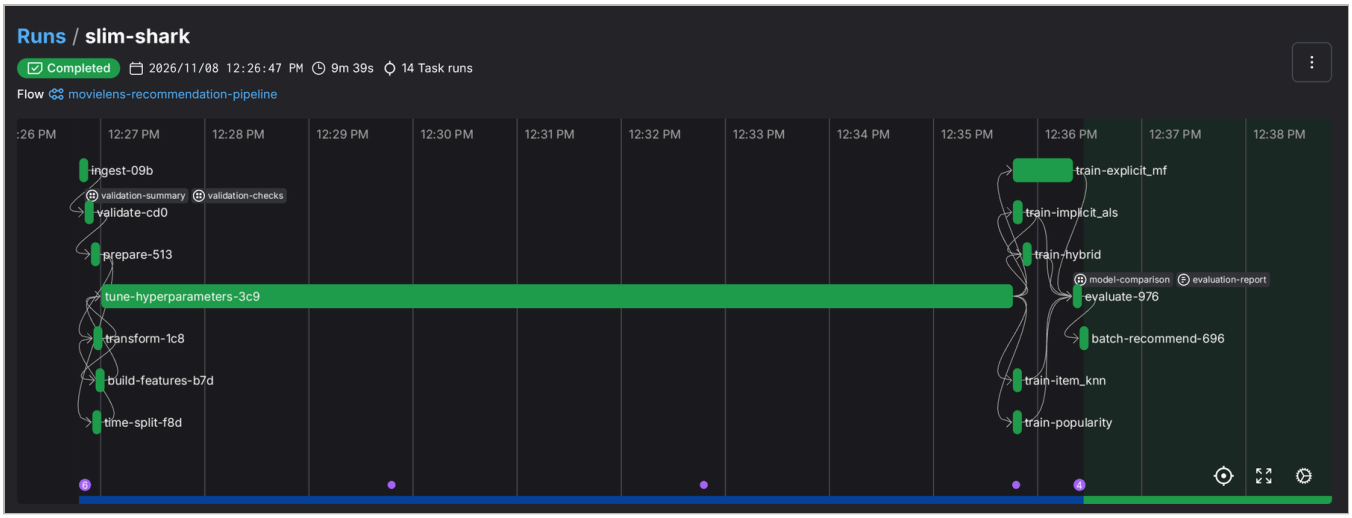
4 LIMITATIONS AND FUTURE WORK

- **Cold start.** Collaborative models cannot score new users or items (29 test users are excluded from evaluation). Serving falls back to popularity; a demographic or content-only model using the store's user features would do better.
- **One temporal split.** The intervals resample users, not time; rolling-origin evaluation would show how stable the ranking of models is across periods. The validation window is small (492 users), so tuning choices within about 2% NDCG are noise.
- **Evaluation bias.** Unrated items count as irrelevant (missing-not-at-random), so absolute precision is understated. Offline metrics are not a substitute for an online A/B test.
- **Data.** The data is old (2000–2003) and clean, so the error path of the validation stage was exercised only with synthetic defects. There is no drift monitoring between runs yet.
- **Infrastructure.** The feature store is batch-only and file-based (no online low-latency store), and MLflow and Prefect run locally without a model registry or CI. Scaling to MovieLens 25M would call for Polars/Spark, the `implicit` library and a database-backed store.

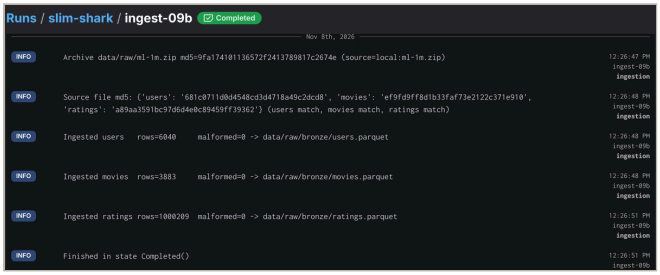
Reproduce: `pip install -r requirements-lock.txt && python -m dags.recommendation_flow` (see README.md). Evidence: `reports/validation/`, `reports/feature_catalog.md`, `reports/evaluation/`, `reports/screenshots/`.

A APPENDIX: SCREENSHOTS OF THE SUCCESSFUL RUN

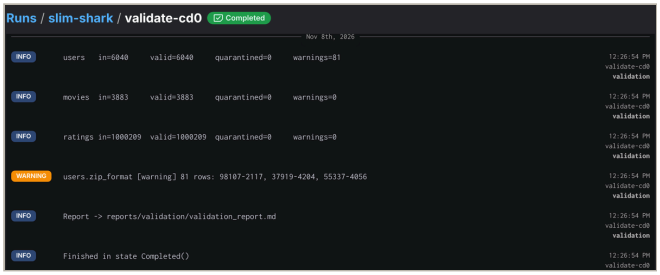
Run evidence, not part of the 3-page report. All screenshots come from one reference run (Prefect flow run *slim-shark*, 14 tasks, all *Completed*). Each stage screenshot is that task's page in the Prefect UI, showing its state and the log lines it wrote. Full-size images are in `reports/screenshots/`.



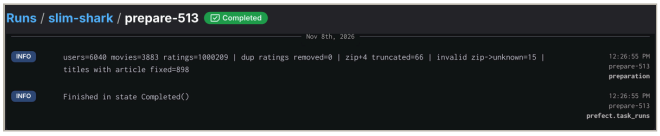
A.1 *Orchestration – the whole flow run: task graph, timeline and status.*



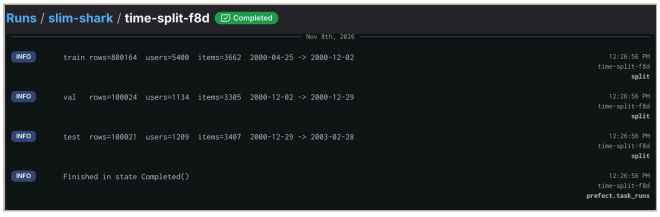
A.2 *Data ingestion – task* `ingest`



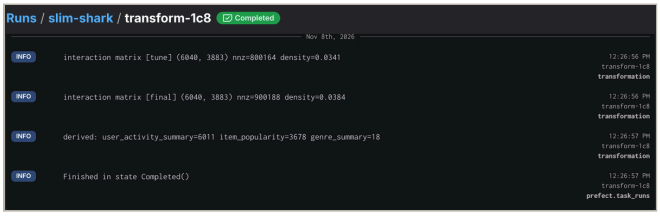
A.3 *Data validation – task* `validate`



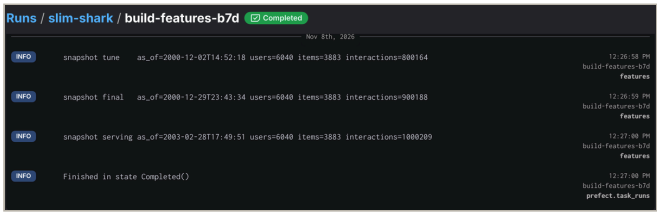
A.4 *Data preparation – task* `prepare`



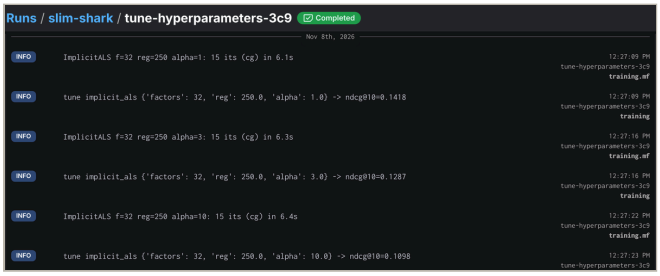
A.5 *Data preparation: time-based split – task* `time-split`



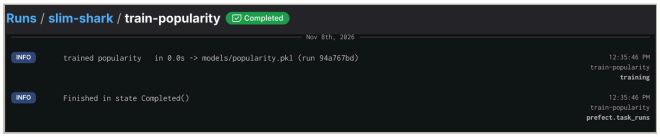
A.6 *Data transformation – task* `transform`



A.7 *Feature management – task* `build-features`



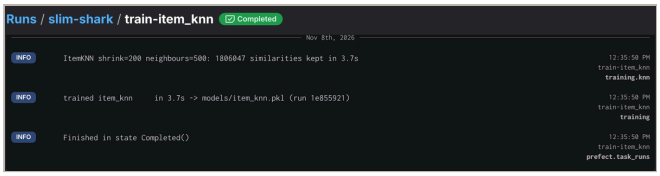
A.8 *Model training: tuning (first of 73 trials shown) – task* `tune-hyperparameters`



A.9 *Model training: popularity – task* `train-popularity`



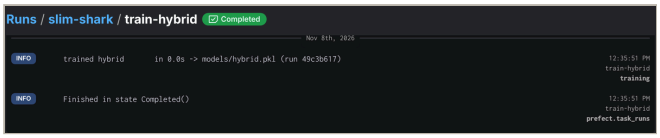
A.10 *Model training: explicit MF – task* `train-explicit_mf`



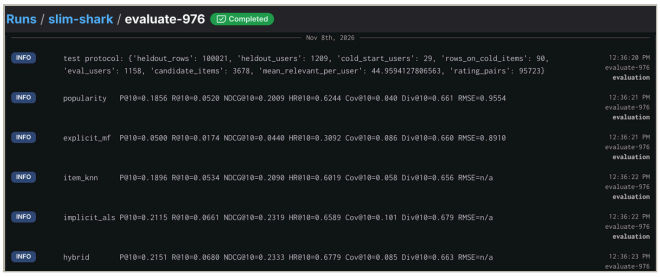
A.11 *Model training: item-kNN – task* `train-item_knn`



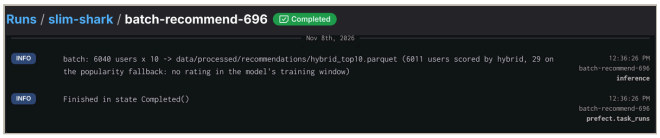
A.12 *Model training: implicit ALS – task* `train-implicit_als`



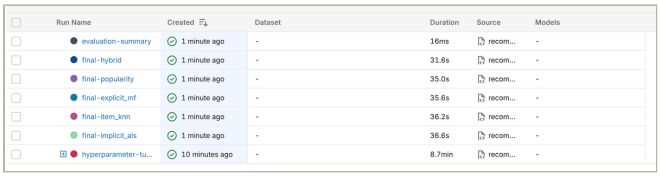
A.13 *Model training: hybrid – task* `train-hybrid`



A.14 *Model evaluation – task* `evaluate`



A.15 *Serving: batch recommendations – task* `batch-recommend`



A.16 *Experiment tracking – the five final models and the tuning parent run in MLflow.*